EXP 6
23/7

# ERROR DETECTION AND CORRECTION USING HAMMER CODE

AIM:

To write a program to implement error detection and correction using hamming code.

ALGROTHM:

* Start the program
* Read the number of 4-bit data blocks (n) forming input
* For each 4-bit data block D = d1, d2, d3, d4 perform Hammer encoding
* Compute the parity bits at position

P1 = d1 XOR d2 XOR d4

P2 = d1 XOR d3 XOR d4

P4 = d2 XOR d3 XOR d4

* At the receiver, recompute the check bits from received code:

C1 = P1 XOR d1 XOR d2 XOR d4

C2 = P2 XOR d1 XOR d3 XOR d4

C4 = P4 XOR d2 XOR d3 XOR d4

* Compute error position
* Extract the original 4 bits
* Repeat step 3 to 8 for all blocks
* Stop the program

PROGRAM:

```
def encode(d):
    h = [0] * 7
    h[2] = d[0]
    h[4] = d[1]
    h[5] = d[2]
    h[6] = d[3]
```

OUTPUT :

```
==HAMMING CODE - ERROR DETECTION
            AND CORRECTION==

Enter number of 4 bit data block
in stream: 2
--- Block 1 ---
Enter 4 data bit (space seperated): 1 0 1 1
Original Data bits: 1011
Encoded Hamming Code: 0110011
Received Code: 0100011
Checking for errors....
   Error detected at bit position 3
   Correcting.....
Corrected Code: 0110011
Recovered data bits: 1011
--- Block 2 ---
Enter 4 data bits: 0110
Original Data bits: 1100110
Encoded Hamming Code: 11000110
Enter bit position: 6
Received Code: 1100100
```

```
def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]
    error_pos = c1*1 + c2*2 + c4*4
    if error_pos != 0:
        printf(f" Error detected at bit:
{error_pos} -> correcting... ")
    else:
        print (" No error detected")
def main():
    print ("== HAMMING CODE - ERROR
          DETECTION AND CORRECTION ==")
    n = int(input("Enter number of 4-bit
                  data block: "))
    h = encode(d)
    printf(" Original data Bits : {to_str(d)}")
    printf(" Encoded Hamming Code (7 bits )")
    for b in range(1, n+1):
        ~~Printf "Original Data bit~~
        d = list(map(int, input("
Enter 4 data bits (space seperated):
).split()))
```

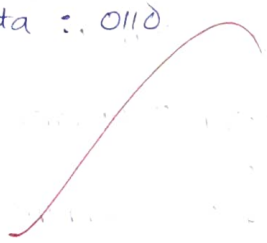
Checking for errors....

Error detected at bit position: 6

Correcting.....

Corrected code : 1100110

Recovered data : 0110



```
if 1 <= pos <= 7:
    h[pos - 1] ^= 1
    print(f"Transmitted code (with error)
else:
    print("Transmitted code (without error)

print("Received code : {to_str(h)}")
print("Checking for error....")
detect_and_correct(h)
print(f" corrected code: {to_str(h)}")
if __name__ == "__main__":
    main()
```

Result:

The program to implement error detection and correction using the hamming code concept was successfully written in python, executed and verified using a sample input data stream.